

Meta-Configuration Files in YOLOv26

Technical Report on YAML, Directory Structure, and Label File Formats

Computer Vision | Object Detection

Abstract — This report provides a concise technical overview of the meta-configuration files that govern dataset management in YOLOv26. It covers the dataset YAML file, standard directory structure, label annotation format, and their collective role in driving model training. The report is intended for practitioners and students beginning work with Ultralytics YOLO frameworks.

1. Dataset YAML Configuration File

The YAML (Yet Another Markup Language) file serves as the primary meta-configuration file in a YOLOv26 dataset. It is a plain-text file that the Ultralytics training engine parses before any data is loaded, making it the single authoritative source of truth for dataset topology and class definitions.

1.1 Structure and Fields

```
# dataset.yaml - annotated example
path: /home/user/datasets/my_dataset      # absolute root of the dataset
train: images/train                       # path relative to 'path'
val: images/val                           # validation split
test: images/test                         # optional held-out split
nc: 3                                     # number of object classes
names:                                    # class index → class name
  - cat      # class 0
  - dog      # class 1
  - person   # class 2
```

The path key holds the absolute path to the dataset root; all relative keys (train, val, test) are resolved against it. The nc field must equal the exact number of entries in the names list — any mismatch raises a runtime error before training begins. The names list is ordered: the integer at each position is the class ID used inside label files. Ultralytics ships ready-made YAML files for benchmark datasets (COCO, VOC, Open Images), which can be used as templates.

2. Dataset Directory Structure

YOLOv26 mandates a strict parallel folder layout: images and their annotation files must reside in mirrored subtrees. The framework resolves label paths automatically by substituting the images prefix with labels and the image extension with .txt.

```
my_dataset/
├── dataset.yaml
├── images/
│   ├── train/  ← JPEG / PNG training images
│   └── val/    ← validation images
├── labels/
│   ├── train/  ← one .txt file per training image
│   └── val/    ← one .txt file per validation image
# Pairing example:
images/train/cat_042.jpg → labels/train/cat_042.txt
```

If a label file is absent for a given image, YOLO treats that image as a hard-negative (background) sample — a valid and common practice for reducing false positives. Deviating from this structure (e.g., placing labels inside the images folder, or using different base names) causes silent data-loading failures that degrade model accuracy without raising explicit errors.

3. Label (Annotation) File Format

Each .txt file contains one line per object instance. Lines follow a fixed five-field format using normalised coordinates:

```
<class_id> <x_center> <y_center> <width> <height>

# All coordinate values are in the range [0.0, 1.0].
# class_id is a zero-based integer matching the names list.
```

3.1 Coordinate Normalisation

Every coordinate is divided by the corresponding image dimension (width or height), making the annotation resolution-independent. A bounding box that occupies the left half of any image is encoded identically regardless of whether the image is 320×240 or 3840×2160.

3.2 Worked Example

```
Image size : 640 × 480 px
Object      : dog, pixel box top-left (100, 80) → bottom-right (300, 320)

x_center = (100 + 300) / 2 / 640 = 0.3125
y_center = ( 80 + 320) / 2 / 480 = 0.4167
width    = (300 - 100) / 640 = 0.3125
height   = (320 - 80) / 480 = 0.5000

Label line (dog = class 1):
1 0.3125 0.4167 0.3125 0.5000
```

Multiple objects in one image are each written on a separate line. Images with no objects are represented by an empty file (or no file at all). This format differs from COCO JSON, which uses absolute pixel coordinates stored in a shared JSON file; conversion utilities (e.g., coco2yolo.py provided by Ultralytics) handle the mapping.

4. Role of Meta-Configuration Files in YOLOv26 Training

The YAML file, directory structure, and label files form a three-layer meta-configuration system. The YAML layer declares what exists (class taxonomy, split locations). The directory layer organises the raw data so the dataloader can iterate efficiently. The label layer encodes where objects are within each frame. Together they control three critical training behaviours:

- Dataset loading — The dataloader reads the YAML to build file lists, then streams image–label pairs in parallel. Incorrect paths produce zero-sample batches, which crash training immediately.
- Model architecture — The nc field directly sets the number of output neurons in the detection head. A mismatch between nc and the actual number of classes in label files causes incorrect loss computation and degraded mean Average Precision (mAP).
- Training stability — Out-of-range normalised coordinates (values > 1.0) or swapped class IDs produce NaN losses and unstable gradient updates. Ultralytics validates label files at the start of training and logs warnings for each malformed entry.

Reproducibility is an additional benefit: storing configuration in a version-controlled YAML file ensures that any collaborator can recreate an identical experiment by updating only the path key to match their local environment.

5. References

[1]	Ultralytics. (2024). YOLOv8 Documentation — Datasets. https://docs.ultralytics.com/datasets/
[2]	Ultralytics. (2024). Train with Custom Data. https://docs.ultralytics.com/guides/
[3]	Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014. https://cocodataset.org/#format-data
[4]	Roboflow. (2024). YOLOv26 PyTorch TXT Format Guide. https://roboflow.com/formats/yolov8-pytorch-txt
[5]	Ultralytics GitHub Repository. https://github.com/ultralytics/ultralytics